NearNest

"Need It. Get It Nearby."

Phase I Project Proposal and Technical Documentation

Subject: NEWAGE

Submitted By: Badal Prasad

Roll Number: 2501351020

Programme: B.Tech FSD

Semester/Section: 5th / A

Under the Guidance of:
Hemraj Kumawat

University:
K.R. Mangalam University (KRMU)

Table of Contents

Section	Topic
1.	Executive Summary & Introduction
2.	Problem Statement & Objectives
3.	Technology Stack & Architecture
4.	System Flow Diagrams (Use Case, DFDs)
5.	Database Design (ERD & Room Schema)
6.	Modules & Implementation Snippets
7.	Testing Strategy
8.	Limitations, Timeline & Conclusion
9.	Viva Questions & Checklist

1. Executive Summary

NearNest is a proposed Minimum Viable Product (MVP) for an Android application aimed at university students and local residents. It facilitates the discovery of nearby essentials, local item borrowing/lending, and localized delivery tasks. This report outlines the Phase I design, technical architecture, and implementation roadmap using modern Android native technologies (Kotlin, Jetpack Compose, Room).

2. Introduction and Project Background

In dense communities such as university campuses, finding specific everyday items (like calculators, chargers, or specialized textbooks) on short notice is challenging. NearNest aims to bridge this gap by offering a local-first mobile interface where users can simulate posting items, requesting urgent needs, and organizing small-scale local delivery tasks.

3. Problem Statement

Current e-commerce applications are optimized for wide-area logistics, lacking the immediacy required for hyper-local campus borrowing or peer-to-peer rapid tasks. There is a distinct lack of a simple, localized platform focusing purely on neighborhood radius transactions without heavy overhead.

4. Motivation and Objectives

Motivation: To leverage modern Android architecture (MVVM) to build a solution that addresses immediate local needs while serving as a comprehensive learning vehicle for B.Tech coursework.

Objectives:

- Design a native Android app using Kotlin and Jetpack Compose.
- Implement local data persistence using Room Database.
- Establish clean architecture utilizing MVVM, Repository pattern, and Coroutines.
- Simulate multiple roles (Customer, Seller, Runner) securely within a local MVP environment.

5. Scope, Existing Approach & Proposed Solution

Scope: The MVP will focus exclusively on local-first functionality. Data will be stored on the device using Room. Cross-device syncing, live payments, and real-time GPS tracking are strictly future enhancements outside the current MVP scope.

Existing Approach: WhatsApp groups or broad marketplace apps (OLX) which are either too unstructured or too broad geographically.

Proposed Solution: A structured Android application with distinct local roles, localized item listings, and demonstration orders that update a local database.

6. Technology Stack and Justification

- **Kotlin:** The modern, expressive language endorsed by Google for Android development.
- **Jetpack Compose:** A declarative UI framework enabling rapid, intuitive UI design without XML layouts.
- **MVVM Architecture:** Separates UI logic from business logic, making code testable and maintainable.
- **ViewModel & StateFlow:** Manages UI-related data reactively, surviving configuration changes like screen rotations.
- **Coroutines & Flow:** Handles background tasks (like DB operations) asynchronously without blocking the main thread.
- **Room Database:** An SQLite object mapping library providing robust local data storage on a single device.
- **JUnit 4 & Coroutines Test:** Frameworks for unit-testing business logic efficiently.

> **Important Constraint Clarification:** Room Database operates *locally* on the user's specific Android device. It does not automatically sync data to the cloud or other users' phones. Any multi-user interactions described in this MVP are simulated via local role-switching. Real cross-device capabilities require a backend server (e.g., REST API), which is slated for future phases.

7. System Architecture (Android Native)

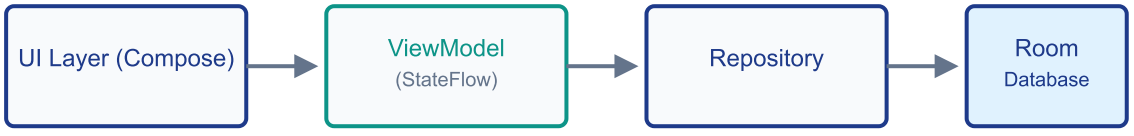


Figure 1: High-Level Android MVVM Architecture Flow

8. Data Flow & Use Cases

Use Case Diagram

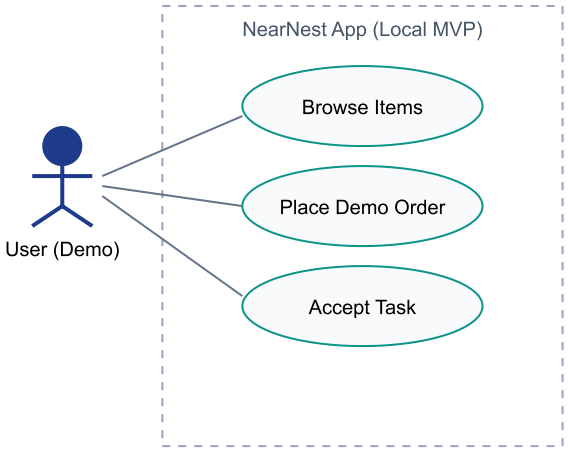


Figure 2: System Use Case Diagram

Data Flow Diagram (Level 0 - Context)

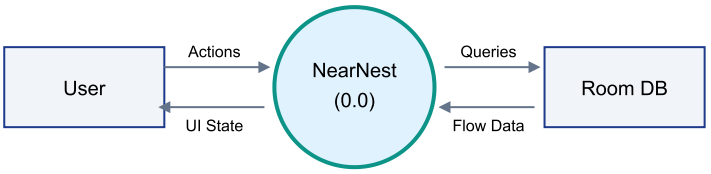


Figure 3: Context Level Data Flow Diagram

9. Entity Relationship Diagram (Local Database)



Figure 4: Simplified ERD mapping to Room Entities

10. Implementation Snippets (Kotlin)

The code below demonstrates how Room Database and Kotlin Coroutines are implemented in NearNest to follow a clean MVVM structure. These beginner-friendly examples highlight local state management.

10.1 Room Entity and DAO

```
import androidx.room.Entity
import androidx.room.PrimaryKey
import androidx.room.Dao
import androidx.room.Insert
import androidx.room.Query
import kotlinx.coroutines.flow.Flow

@Entity(tableName = "items")
data class Item(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val name: String,
    val description: String,
    val price: Double
)

@Dao
interface ItemDao {
    @Insert
    suspend fun insertItem(item: Item)

    // Flow emits real-time updates when the database changes
    @Query("SELECT * FROM items")
    fun getAllItems(): Flow<List<Item>>
}
```

10.2 Repository and ViewModel

```
import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import kotlinx.coroutines.launch
import kotlinx.coroutines.flow.SharingStarted
import kotlinx.coroutines.flow.stateIn

class ItemRepository(private val itemDao: ItemDao) {
    val allItems = itemDao.getAllItems()
    suspend fun addItem(item: Item) = itemDao.insertItem(item)
}

class MainViewModel(private val repository: ItemRepository) : ViewModel() {
    // StateFlow holds state and survives screen rotations
    val itemsState = repository.allItems.stateIn(
        scope = viewModelScope,
        started = SharingStarted.WhileSubscribed(5000),
        initialValue = emptyList()
    )

    fun addNewItem(name: String, desc: String, price: Double) {
        viewModelScope.launch {
            repository.addItem(Item(name = name, description = desc, price = price))
        }
    }
}
```

11. Testing Strategy

Unit tests will be written using JUnit 4 and `kotlinx-coroutines-test` to ensure that ViewModel logic correctly processes UI actions. Simulated testing ensures application stability without requiring physical device farms.

12. Security, Limitations & Future Enhancements

- **Limitations:** Currently, "Authentication" is a simulated role-selection screen. No real user verification is performed. All data is locked to the specific hardware device.
- **Future Enhancements:** Migration to a cloud backend (e.g., Firebase or REST API) for multi-device synchronization; integration of secure JWT authentication; addition of Google Maps API for real-time delivery tracking.

13. Viva Questions & Expected Answers

Q. No.	Question	Short Answer
1	What is NearNest?	A local-first Android app for borrowing and delivering nearby essentials.
2	Why choose Kotlin over Java?	Kotlin is concise, null-safe, and officially recommended by Google.
3	What is Jetpack Compose?	A modern, declarative UI toolkit for building native Android interfaces.
4	Explain MVVM.	Model-View-ViewModel architecture separates UI (View) from data (Model) using a ViewModel.
5	Does Room DB sync across devices?	No, Room is a local SQLite abstraction. It stores data only on the user's phone.
6	What are Kotlin Coroutines?	Lightweight threads used for asynchronous programming, like DB calls.
7	Why use StateFlow?	It holds UI state and updates the View reactively when data changes.
8	What is the Repository Pattern?	A layer that acts as a single source of truth, managing data operations.
9	How are dependencies managed?	Using Gradle Kotlin DSL (build.gradle.kts).
10	What does @Entity do?	It marks a Kotlin data class as a database table in Room.

14. Official Documentation References

1. Android Developers: https://developer.android.com/
2. Kotlin Documentation: https://kotlinlang.org/docs/home.html
3. Jetpack Compose UI: https://developer.android.com/develop/ui/compose
4. Room Local Database: https://developer.android.com/training/data-storage/room

15. Conclusion

The Phase I proposal for NearNest demonstrates a practical application of the NEWAGE B.Tech curriculum. By utilizing industry-standard Android technologies like Compose and Room, the project ensures a solid foundation for building a robust, locally functioning MVP.